

4. SQL 로그 캡처

BoardMapper.xml 의 <where>,<if>,<foreach>가 실제 SQL로 어떻게 바뀌는지 로그 캡처

```
curl -s 'http://localhost:18080/api/boards/search?'
```

파라미터를 붙이지 않았을 때 where 절을 타지 않아서 where 절이 보이지 않고 전체 데이터가 보이는 것을 알 수 있다.

```
Registering transaction synchronization for SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3de72a37]
JDBC Connection [HikariProxyConnection@656502919 wrapping org.mariadb.jdbc.Connection@6c7ab2ba] will be managed by Spring
=> Preparing: SELECT id, title, content, writer, views, created_at FROM board ORDER BY id DESC LIMIT 20
=> Parameters:
== Columns: id, title, content, writer, views, created_at
== Row: 5, REST API 설계 예고, 6주차 RESTful API 설계로 이어집니다., donghun, 1, 2026-05-17 22:12:18
== Row: 4, 조회수 증가 동시성, views = views + 1 방식으로 증가합니다., mentor, 10, 2026-05-16 22:12:18
== Row: 3, Cursor 페이징 테스트, id 기준 cursor 페이징을 실습합니다., donghun, 0, 2026-05-15 22:12:18
== Row: 2, MyBatis foreach 예제, IN 절과 빈 리스트 방어를 확인합니다., mentor, 7, 2026-05-14 22:12:18
== Row: 1, Spring Boot 동적 쿼리 정리, where, if, foreach 실습용 게시글입니다., donghun, 3, 2026-05-13 22:12:18
== Total: 5
```

```
curl -s 'http://localhost:18080/api/boards/search?title=Cursor&writer=donghun'
```

파라미터 title과 writer를 붙여서 요청을 보내니 쿼리문이 if 문 안에 있는 title은 AND를 지우고 WHERE 절로 사용되었고 writer는 AND를 붙여서 사용되었다.

```
Registering transaction synchronization for SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@34517a90]
JDBC Connection [HikariProxyConnection@1683417387 wrapping org.mariadb.jdbc.Connection@13e4acbb] will be managed by Spring
=> Preparing: SELECT id, title, content, writer, views, created_at FROM board WHERE title LIKE CONCAT('%', ?, '%') AND writer = ? ORDER BY id DESC LIMIT 20
=> Parameters: Cursor(String), donghun(String)
<== Columns: id, title, content, writer, views, created_at
<== Row: 3, Cursor 페이징 테스트, id 기준 cursor 페이징을 실습합니다., donghun, 0, 2026-05-15 22:12:18
<== Total: 1
```

```
curl -s 'http://localhost:18080/api/boards/bulk?ids=1&ids=3&ids=5'
```

중요 포인트: ids=1&ids=3&ids=5 가 List 으로 바인딩 SQL 로그에 WHERE id IN (?, ?, ?) 형태가 나오는지 확인 응답 순서는 SQL의 ORDER BY id DESC 때문에 5, 3, 1

```

Releasing transactional SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3de72a37]
Transaction synchronization committing SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3de72a37]
Transaction synchronization deregistering SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3de72a37]
Transaction synchronization closing SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3de72a37]
Creating a new SqlSession
Registering transaction synchronization for SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@148588dd]
JDBC Connection [HikariProxyConnection@532026612 wrapping org.mariadb.jdbc.Connection@6c7ab2ba] will be managed by Spring
==> Preparing: SELECT id, title, content, writer, views, created_at FROM board WHERE id IN ( ?, ?, ? ) ORDER BY id DESC
==> Parameters: 1(Long), 3(Long), 5(Long)
<== Columns: id, title, content, writer, views, created_at
<== Row: 5, REST API 설계 예고, 6주차 RESTful API 설계로 이어집니다., donghun, 1, 2026-05-17 22:12:18
<== Row: 3, Cursor 페이징 테스트, id 기준 cursor 페이징을 실습합니다., donghun, 0, 2026-05-15 22:12:18
<== Row: 1, Spring Boot 동적 쿼리 정리, where, if, foreach 실습용 게시글입니다., donghun, 3, 2026-05-13 22:12:18
<== Total: 3

```

5. EXPLAIN 결과 캡처

SELECT id, title, content, writer, views, created_at FROM board WHERE title LIKE
CONCAT('%', ?, '%') AND writer = ? ORDER BY id DESC LIMIT 20

Cursor(String), donghun(String)

Type = ref

의미: 인덱스를 타고 있다.

board (1r x 10c)										
#	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	board	ref	idx_board_writer	idx_board_writer	202	const	3	Using where

6. Offset vs Cursor 차이 정리

일단 내가 이해한 바로는 Offset 형식은 특정 페이지로 바로 이동이 가능하다는 장점이 있고 단점으로는 데이터가 지속적으로 추가되거나 삭제되는 환경에서 페이지네이션 중 데이터의 중복이나 누락이 발생할 수 있습니다.

Cursor 방식은 동적으로 데이터가 추가되거나 삭제되는 상황에서 특정 데이터 요소를 기준으로 하기 때문에 조회 중 데이터 변경이 발생해도 영향을 받지 않습니다.

단점으로는 사용자가 원하는 페이지로 바로 이동할 수 없습니다.

이유는 cursor기반 페이지네이션은 총 데이터의 개수를 알 수 없어, 원하는 페이지로 바로 이동하는 것이 불가능합니다.

7. 조회수 증가 위험한 방식 vs 안전한 방식 정리

조회수 증가에 `views++`를 사용하면 `read → +1 → write`라서 동시에 실행되면 `lost update`가 생긴다는 점이 위험한 방식이고

안전한 방식으로는

```
UPDATE board
```

```
SET views = views + 1
```

```
WHERE id = #{id}
```

이런식으로 Database Level에서 원자적 연산을 합니다.

- 애플리케이션은 계산을 하지 않고, DB에게 "현재 값에 1을 더해라"라는 명령어만 던집니다.
- **안전한 이유: 데이터베이스의 원자성(Atomicity)과 락(Locking)**

RDBMS(MySQL, Oracle 등)는 같은 행(Row)을 업데이트할 때 내부적으로 배타락(Exclusive Lock, 쓰기락)을 사용합니다.

사용자 A와 B가 동시에 이 쿼리를 날리더라도, DB 내부에서는 순서가 강제됩니다.

1. 사용자 A의 쿼리가 먼저 실행되면서 해당 행에 락을 겁니다.
2. 사용자 B의 쿼리는 A의 작업이 끝날 때까지 대기합니다.
3. A가 조회수를 101로 올리고 락을 해제하면, 대기하던 B가 업데이트된 101을 기준으로 다시 +1을 수행하여 최종적으로 102가 됩니다.

한 걸음 더 나가기: 이 방식도 만능은 아닙니다 (성능 한계)

직접 `views = views + 1`을 하는 방식은 **데이터 정합성(정확도)** 측면에서는 완벽하지만, 대규모 서비스(예: 인기 연예인의 게시글, 실시간 뉴스)에서는 성능 병목(Bottleneck)이 발생할 수 있습니다.

너도나도 같은 행(Row)을 업데이트하려고 줄을 서기 때문(Lock 대기 상태)에 DB 전체가 느려질 수 있죠.

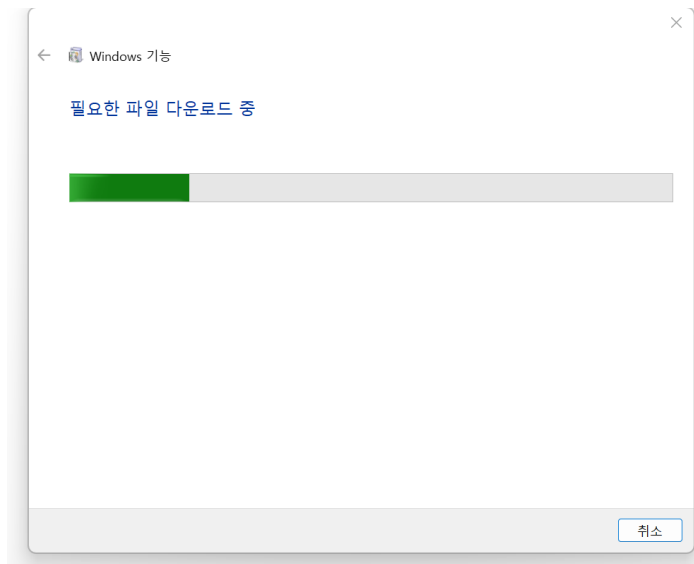
실무에서의 대안: 아주 트래픽이 높은 서비스에서는 매번 DB를 직접 치지 않고, **Redis** 같은 고성능 인메모리 DB의 `INCR` 명령어(이것 역시 원자적 연산이라 안전함)를 이용해 조회수를 빠르게 올린 뒤, 일정 주기(예: 5분마다)로 DB에 몰아서 한 번에 업데이트(벌크 업데이트)하는 방식을 주로 사용합니다.

8. 막힌 부분 또는 더 풀어 듣고 싶은 부분

현재 막힌 부분은 도커 사용이 막혀서 찾아보고 있습니다. 윈도우에서 도커를 사용하려고 하는 wsl 업데이트를 해야한다해서 직접 설치를 해봤는데 wsl 설치 후에 도커를 완전 삭제하고 다시 해보려하니 도커 다운이 제대로 안되고 있습니다.

해본 방식

1. 도커 완전 삭제 예시 경로를 들어가서 도커 관련된 파일을 다 삭제해줬습니다.
2. WSL 문제일수도 있어서 WSL을 다시 삭제하고 하려 했지만 정확히는 모르겠어서 우선 놔두었습니다.
3. 윈도우 기능 켜기에서 가상 머신 플랫폼을 켜줘야 한다고 들어서 클릭하고 확인을 누르면



이런식으로 나오다가 혼자서 멈춰버립니다.

제 생각으로는 도커와 관련해서 무엇이 고인거 같은데 그게 정확히는 모르겠어서 제 나름 해보고는 있는데 잘 안되네요....

그래서 위 내용들은 도커로 하지 않고 heidsql에서 데이터를 그냥 만들어서 테스트 해보고 제출한 내용입니다.